

Dart Programming

- An enum is a special type that represents a fixed number of constant values. An enum is declared using the keyword `enum` followed by the enum's name.
- `enum enumName {`
 - `constantName1,`
 - `constantName2,`
 - `constantName3,`
 - `...`
 - `constantNameN`
- `}`

Example

- enum days {
- Sunday,
- Monday,
- Tuesday,
- Wednesday,
- Thursday,
- Friday,
- Saturday
- }

```
void main() {  
    var today = days.Friday;  
    switch (today) {  
        case days.Sunday:  
            print("Today is Sunday.");  
            break;  
        case days.Monday:  
            print("Today is Monday.");  
            break;  
        case days.Tuesday:  
            print("Today is Tuesday.");  
            break;  
        case days.Wednesday:  
            print("Today is Wednesday.");  
            break;  
        case days.Thursday:  
            print("Today is Thursday.");  
            break;  
        case days.Friday:  
            print("Today is Friday.");  
            break;  
        case days.Saturday:  
            print("Today is Saturday.");  
            break;  
    }  
}
```

```
enum Gender { Male, Female, Other }

class Person {
    // Properties
    String? firstName;
    String? lastName;
    Gender? gender;

    // Constructor
    Person(this.firstName, this.lastName, this.gender);

    // display() method
    void display() {
        print("First Name: $firstName");
        print("Last Name: $lastName");
        print("Gender: $gender");
    }
}

void main() {
    Person p1 = Person("John", "Doe", Gender.Male);
    p1.display();

    Person p2 = Person("Menuka", "Sharma", Gender.Female);
    p2.display();
}
```

- Advantages Of Enum In Dart

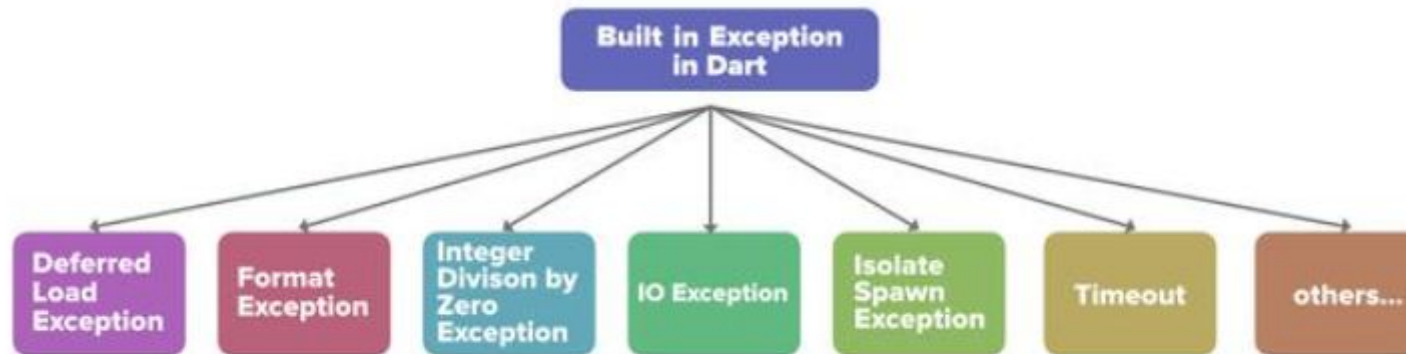
- It is used to define a set of named constants.
- Makes your code more readable and maintainable.
- It makes the code more reusable and makes it easier for developers.

- Characteristics Of Enum

- It must contain at least one constant value.
- Enums are declared outside the class.
- Used to store a large number of constant values.

Exception Handling

- An exception is an error that takes place inside the program.
- When an exception occurs inside a program the normal flow of the program is disrupted and it terminates abnormally, displaying the error and exception stack as output. So, an exception must be taken care to prevent the application from termination.



Exception Handling

- Every built-in exception in Dart comes under a pre-defined class named Exception.
- To prevent the program from exception we make use of try/on/catch blocks in Dart.
- try {
 - // program that might throw an exception
 - }
 - on Exception1 {
 - // code for handling exception 1
 - }
 - catch Exception2 {
 - // code for handling exception 2
 - }

```
• void main() {  
•   String message = "Hello";  
•   try{  
•     var test = message ~/ 0;  
•     print(test);  
•   }  
•   on FormatException{  
•     print("Error!! \nCan't act as input is not an integer.");  
•   }  
• }
```

Example

- void main() {
- String message = "Hello";
- try{
- var test = message ~/ 0;
- print(test);
- }
- catch(e){
- print("e");
- }
- }

- Final block: The final block in dart is used to include specific code that must be executed irrespective of error in the code.
- Although it is optional to include finally block if you include it then it should be after try and catch block are over.

```
• void main() {  
•   int n = 10;  
•   try{  
•     var n2 = n ~/ 0;  
•     print(n2);  
•   }  
•   catch(e){  
•     print(e);  
•   }  
•   finally {  
•     print("Code is at end");  
•   }  
• }
```

- Unlike other languages, in Dart to one can create a custom exception. To do, so we make use of throw new keyword in the dart.
- // extending Class Age
- // with Exception class
- class Age implements Exception {
- String error() => 'your age is less than 18 :(';
- }

Example

```
• void main() {  
•   int age1 = 20;  
•   int age2 = 10;  
•   try {  
•     check(age1);  
•     check(age2); }  
•   catch(e){  
•     // Printing error  
•     print(e.error());  
•   } }  
•   // Checking Age  
•   void check(int age){  
•     if(age < 18){  
•       throw new Age(); }  
•     else {  
•       print("You are eligible to visit :)");  
•     }  
•   }
```

Throwing Exceptions

- The throw keyword is used to explicitly raise an exception. A raised exception should be handled to prevent the program from exiting abruptly.
- The syntax for raising an exception explicitly is – `throw new Exception_name()`

```
• main() {  
•   try {  
•     test_age(-2);  
•   }  
•   catch(e) {  
•     print('Age cannot be negative');  
•   }  
• }  
• void test_age(int age) {  
•   if(age<0) {  
•     throw new FormatException();  
•   }  
• }
```

Q & A time